# Exercise 2: MATLAB preliminaries for IN3015/IN4015

Håvard Kjellmo Arnestad (haavaarn@ifi.uio.no)

August 26, 2022

## Introduction

In this first part of module 1 we will familiarize ourselves with some MATLAB concepts that will come in handy later in the course. This exercise will only go into a few of the details of MATLAB programming, and it is wise to use this opportunity to verify and/or learn what you need in order to be comfortable with basic scientific programming in MATLAB.

The best tip in general is to use the internet actively to find what you need. This includes YouTube, StackOverflow, and notably the documentation provided by MathWorks (the company behind MATLAB). It is well worth spending some time getting familiar with how functions (e.g., `sum`: `https://se.mathworks.com/help/matlab/ref/sum.html`) are documented.

The experience from earlier years in IN3015/IN4015 is that students have varied backgrounds. Ideally you should know a programming language already, and for many this will be Python. The "scientific stack" in Python includes Numpy and Scipy, which is very similar to MATLAB. Two relevant resources for those with a Python background are *Numpy for Matlab users* at
`https://numpy.org/doc/stable/user/numpy-for-matlab-users.html`
or *Matlab for Python users* at
`https://blogs.mathworks.com/student-lounge/2021/02/19/introduction-to-matlab-for-python-users/`

MathWorks have a lot of official resources, for instance this *getting started* page:
`https://se.mathworks.com/help/matlab/getting-started-with-matlab.html`
An alternative is the onramp tutorial that starts from the very basics, and lets you program interactively in the web browser. Note that you do need to be logged in:
`https://matlabacademy.mathworks.com/details/matlab-onramp/gettingstarted`

## Within MATLAB

In Fig. 1 we can see the MATLAB user interface. The left pane is the "current folder" which shows us what files MATLAB has access to. One of these functions is the `test_function.m` that is also shown in the "editor". This function takes in a number $x$, and returns a complex number $x + 2ix$. The "command window" allows us to run this function, and we assign the output to the variable `a`. This variable is now known to MATLAB as we can see it in the "workspace", but when we can use it depends on the *scope* (scope determines where in your program a name is visible).

In many cases, such as when we are working with USTB, our functions are not located in our current folder. We then need to make sure the functions are on the MATLAB "path". MATLAB uses the search path to locate

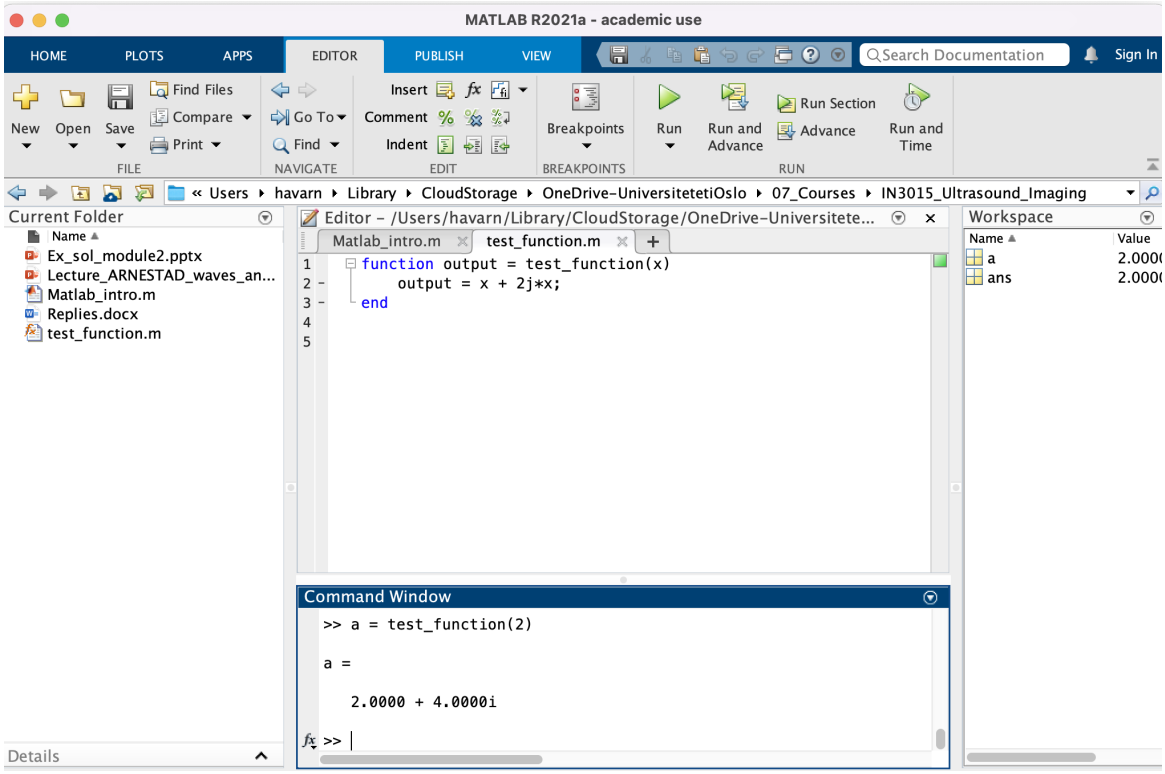


Figure 1: The MATLAB interface.

files in the system. You can include folders from anywhere to the path by navigating to them, right-clicking, and selecting add to path, or by using the `addpath` function.

# TASKS

In the `MATLAB_intro.m` file, you will find sections of code (separated by `%%`) that can be run individually. Run the code and answer the questions in the file.

## Hints: 1) Arrays - lists of numbers

**a)** In MATLAB 1D-arrays are either row (lying) or column (standing) vectors. Type `x_row` into the command window and press enter. What about `x_col`?

**b)** The fact that MATLAB separates between row and column vectors makes it important to be aware of pitfalls when, e.g., multiplying two arrays together. How well do you remember your linear algebra? Some combinations output a matrix! Keywords: inner product (dot product), outer product (sometimes: tensor product), element-wise product (Hadamard product).

**c)** If you multiply an array with a complex number, all entries are also multiplied.

**d)** In MATLAB there is a difference between the transpose and the conjugate transpose. This can be hard to debug if you get it wrong the first time!

### Hints: 2) Multidimensional arrays

In the next few modules we will find that the datasets in ultrasound imaging are multidimensional: (time x elements x transmit direction). This is one more dimension than matrices and therefore harder to visualize.

**a)** Matrices can be composed as rows of columns, or columns as rows (remember the difference between commas "," and semicolons ";"). However, there are other methods that are often employed for structured matrices such as `ones()`, `eye()`, `magic()`, `diag()`.

**b)** In addition to using `size()` it is often useful to inspect the object or variable in the workspace. Double click on `B` to get a more detailed view.

**c)** Looking at the documentation is always useful: `https://se.mathworks.com/help/matlab/ref/sum.html`.

**d)** No hints here.

## Other tips / optional

`ginput(n)` - This function lets you identify the coordinates of $n$ points in a plot with the cursor. Try to calculate the period of the sinusoid. Can you get close to $T = 3$?

```
x = 0:0.1:10;
T = 3;
y = sin(2*pi/T*x);
plot(x,y); ylim([-2,2]); xlabel('x'); ylabel('y');
[px, py] = ginput(2);
disp( abs(px(2)-px(1)) )
```

At some point you will certainly run into bugs that are hard to fix. Learning to use MATLABs debug tools are very helpful. Writing `dbstop if error` is one way to set breakpoint when something goes awry. In nested function calls it can be difficult to asses why an error is raised, but from the toolbar during debug mode you can always access any layer of the "Function Call Stack". Write code that raises an error, and see if you can find the bug using the built-in tools.